CHAPTER 1

DESIGNING AN ONLINE STOCK BROKERAGE SYSTEM

1 Getting Ready: An Online Stock Brokerage System 2
2 Requirements for the Online Stock Brokerage System 3
3 Use Case Diagram for the Online Stock Brokerage System 4
4 Class Diagram for the Online Stock Brokerage System 7
5 Sequence Diagram for the Online Stock Brokerage System 20
6 Activity Diagram for the Online Stock Brokerage System 20
7 Code for the Online Stock Brokerage System 21
8 Summary: Designing an Online Stock Brokerage System 29
9 Key Insights and Tips: Designing a Stock Brokerage System 32

This chapter covers the design of an online stock brokerage system, including requirements, UML diagrams, and implementation code.

1 Getting Ready: An Online Stock Brokerage System

1.1 Problem definition

An **online stock brokerage** system acts as an intermediary between the buyer and seller during the trade of stocks. The system facilitates its users' buying and selling of stocks online. It enables users to monitor and carry out their transactions and displays performance graphs for the various stocks in their portfolios. It also protects client transactions and notifies them when stock changes reach certain levels.
The online stock brokerage system streamlines traditional trading using digital infrastructure, offering faster transactions and reduced operational costs. It also offers users quicker access to stock information, market trends, and real-time prices. Additionally , the system supports deposits and withdrawals through checks, wire transfers, or electronic bank transfers, enhancing overall financial flexibility for users.

Figure 1.1: The online stock brokerage system

1.2 Expectations from the interviewee

An online stock brokerage system consists of multiple components. Each component has its own functionality and constraints. The following section provides an overview of some of the main expectations the interviewer will want to hear you discuss in more detail during the interview.

Discoverability

For an online stock brokerage system, discoverability is one of the key features. You can ask the following questions to know more about the system:

- How do the members search the stock inventory?
- How will the search surface result?

Visibility

To get a better understanding of how the data is visible to the different users, you may ask the following questions:

- Can every member see the current levels of stock positions at any time?

Order type

You may ask the interviewer about the types of orders the system should be able to handle by simply asking the question listed below:

- How many types of stock trade orders can the users place, for example, a market order, loss order, etc?

Multiplicity

You may ask the interviewer questions related to the multiplicity of the system. These questions are listed below:

- Can the members have multiple watchlists containing multiple stock quotes?
- Can a member buy multiple lots of the same stock at different times?

1.3 Design approach

We'll design this online stock brokerage system using the bottom-up design approach. For this purpose, we will follow the steps below:

- Identify and design simple components, like stock and stock position, first.
- Use these small components to design bigger components, such as the order and stock inventory, which can be composed of multiple stock items.
- Repeat the steps above until we design the whole stock brokerage system.

1.4 Design pattern

During an interview, discussing the design patterns that an online stock brokerage system falls under is always a good practice. Stating the design patterns gives the interviewer a positive impression and shows that the interviewee is well-versed in the advanced concepts of object-oriented design.
Try to answer the following question. If you are not familiar with design patterns, don't worry! You can learn about them by asking "Define design patterns."

Question:

Which design pattern(s) should be used to design an online stock brokerage system? Please elaborate on your choice(s).

Let's explore the requirements of the online stock brokerage system in the next lesson.

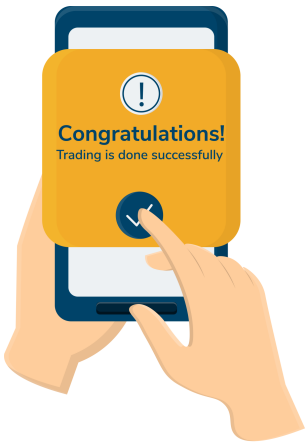
2 Requirements for the Online Stock Brokerage System

In this lesson, we'll list the requirements of the online stock brokerage system. This is a crucial step as requirements define the scope of a problem. Getting them right from the interviewer and understanding them well will make the design of the rest of the system smooth and easy.
We'll use the notational convention to identify each requirement with a unique label "Rn," where "R" is short for requirement and "n" is a natural number.

2.1 Requirement collection

The following are the requirements that we have defined for the online stock brokerage system:

R1: The system should allow the user to easily trade in stocks (buy or sell the stocks).
R2: Users can have numerous watch lists consisting of different stock quotes.
R3: Users may own different lots of the same stock. If a user has purchased the same stock more than once, the system should be able to distinguish between several lots of it.
R4: The system should be able to notify users whenever a trade order is executed.



R5: The system should allow the user to order the stock trade of the types given below:

- **Market order:** Buy or sell stocks at the current market price.

- **Limit order:** Buy or sell stocks at the price set by the user.
- **Stop-loss order:** Buy or sell stocks when they reach a certain price.
- **Stop-limit order:** Buy or sell stocks with a restriction on the limit price (maximum price to be paid, minimum price to be received, etc).

R6: The system should allow users to make deposits and withdrawals using checks, wire transfers, or electronic bank transfers.

We've identified our requirements for the problem, and in the next lesson, we will define different use cases for the online stock brokerage system.

3 Use Case Diagram for the Online Stock Brokerage System

Let's build the use case diagram of the online stock brokerage system and understand the relationship between its different components. First , we'll define the different elements of our online stock brokerage, followed by the complete use case diagram of the system.

3.1 System

Our system is a stock brokerage.

3.2 Actors

Now, we'll define the main actors of our online stock brokerage system.

Primary actor

- **Member:** The member is the main user of the system who can create, update, or cancel an account, log in and out, and reset their password. Members can search for stocks, manage watch lists, view stock positions, and place different orders (market, limit, stop-loss, and stop-limit). They can select stock details, cancel orders, and perform transactions via electronic transfer, wire, or check. Members also receive notifications about order and transaction statuses.

Secondary actors

- **Admin:** The admin manages user accounts. They can create, update, or cancel accounts, log in and out, reset passwords, and block or unblock members when necessary.

- **System:** The system handles internal processes such as fetching stock quotes, sending order details to the stock exchange, acknowledging and processing orders, and notifying members about order and transaction updates.

3.3 Use cases

This section will define the use cases for the online stock brokerage system. We have listed the use cases according to their interactions with a particular actor.

> **Note:** Some use cases will occur multiple times because they are shared among different actors in the system.

Member

- **Create account:** To register a new user in the system.
- **Update account:** To modify existing account information.
- **Cancel membership:** To terminate the user's membership and access.
- **Login/Logout:** To authenticate or end a user session.
- **Reset password:** To recover access by setting a new password.
- **Search stock inventory:** To look up stock details using a symbol.
- **View stock positions:** To display current holdings and quantities.
- **Create watch list:** To create a new list to track favorite stocks.
- **Update watch list:** To add or remove stocks from an existing watchlist.
- **Place order:** To submit an order to buy or sell stocks.
- **Select stock details:** To choose stock, order type, quantity, and price..
- **Cancel order:** To stop a pending order before execution.
- **Deposit money:** To fund the account via electronic transfer, wire, or check.
- **Withdraw money:** To transfer funds out of the trading account.
- **Receive order notification:** To get alerts on order execution status.
- **Receive transaction notification:** To be informed about deposit or withdrawal updates.

Admin

- **Create account:** To add a new member to the system.
- **Update account:** To modify existing member details.
- **Cancel membership:** To terminate a member's access to the system.
- **Login/Logout:** To access or exit the admin interface.
- **Reset password:** To assist a member in password recovery.

System

- **Fetch stock quotes:** To retrieve live stock prices from the exchange.
- **Send order details:** To forward order information to the stock exchange.
- **Execute order:** To process, match, and fulfill stock orders.
- **Deduct stock:** To adjust inventory after trade execution.
- **Send order notification:** To notify members of order status changes.
- **Send transaction notification:** To inform members of deposit or withdrawal outcomes.

3.4 Relationships

This section describes the relationships between and among actors and their use cases.

Generalization

- The "Electronic bank transfer," "Wire transfer," and "Check transfer" use cases are used for transactions. Hence , they have a generalization relationship with the "Transaction" use case.
- The "Place market order," "Place limit order," "Place stop-loss order," and "Place stop-limit order" use cases represent different ways of placing a trade. Hence , they have a generalization relationship with the "Place order" use case.

Associations

The table below shows the association relationship between actors and their use cases.

Member	Admin	System
Create account	Create account	Fetch stocks quotes
Update account	Cancel membership	Send order details
Cancel membership	Update account	Send deposit /withdrawal status change notification
Login/Logout	Login/Logout	Send notification
Reset password	Reset password	Execute order
Search stock inventory		Deduct stock
View stock positions		
Create watchlist		
Update watchlist		
Place order		
Select stock details		
Cancel order		
Deposit money		
Withdraw money		

Include

When a user places an order, they must provide all relevant details such as stock, order type, quantity, price/limit, and time enforcement.

- The "Place order" use case includes the "Select stock," "Select order type," "Select quantity," and "Select price/limit"use cases.

When a user places an order, the system processes stock deduction, sends notifications, and forwards order details.

- The "Place order" use case includes the "Deduct stock," "Send notification," and "Send order details" use cases.

The system retrieves the latest stock quotes when a user selects stock details.

- The “Select stock details” use case includes the “Fetch stock quotes” use case.

When a user deposits or withdraws money, the system sends a notification.

- The “Deposit/Withdraw money” use case includes the “Send notification” use case.

3.5 Use case diagram

Here’s the use case diagram of the online stock brokerage system:

Figure 1.2: The use case diagram of the online stock brokerage system

In the next lesson, we will discuss the class diagram with a detailed explanation of all classes and their relationship with each other.

4 Class Diagram for the Online Stock Brokerage System

In this lesson, we’ll identify and design the classes, abstract classes, and interfaces based on the requirements we have previously gathered from the interviewer in an online stock brokerage system.

4.1 Components of a stock brokerage system

As mentioned, we will design the online stock brokerage system using a bottom-up approach.

Account

`Account`: This abstract class stores a person’s account information. It has members like account ID, name, password, account status, address, email, and phone number. There can be two types of accounts: `Admin`

and `Member`.
`Member`**:** They can search the stock, place an order to buy or sell stocks, create an account, start a membership, add stocks to the wish list, add buying and selling limits, and perform transactions in many ways.
`Admin`**:** They can block or unblock members, cancel their membership, and reset passwords.

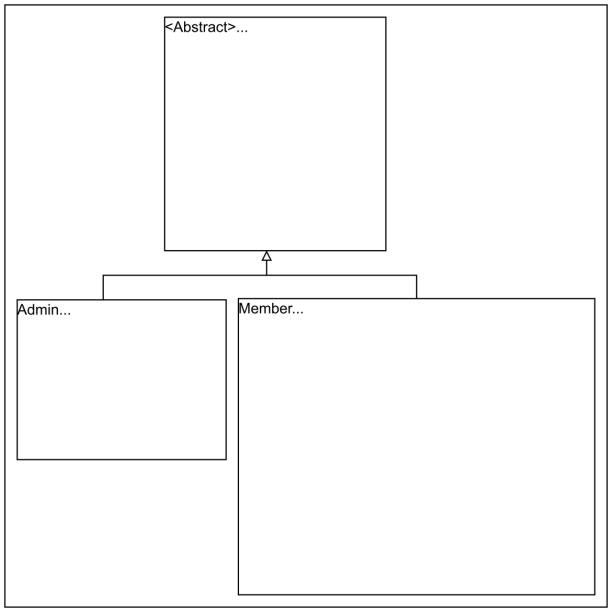


Figure 1.3: The class diagram of Account and its derived classes

Watch list

A **watch list** is a list of stocks that an investor monitors to profit from price drops. Below is a visual representation of the `Watchlist` class.

Figure 1.4: The class diagram of the Watchlist class

R2: Users can have numerous watch lists consisting of different stock quotes.

Stock

A **stock**, also known as equity, is a security that represents a portion of the issuing company's ownership. The `Stock` class will have a symbol, price, etc.

R1: The system should allow users to easily trade stocks (buy or sell them).

Figure 1.5: The class diagram of the Stock class

Search and stock inventory

The `StockInventory` class will retrieve and keep up with the most recent stock values from the `StockExchange` class (defined later). The `StockInventory` class implements the `Search` interface.

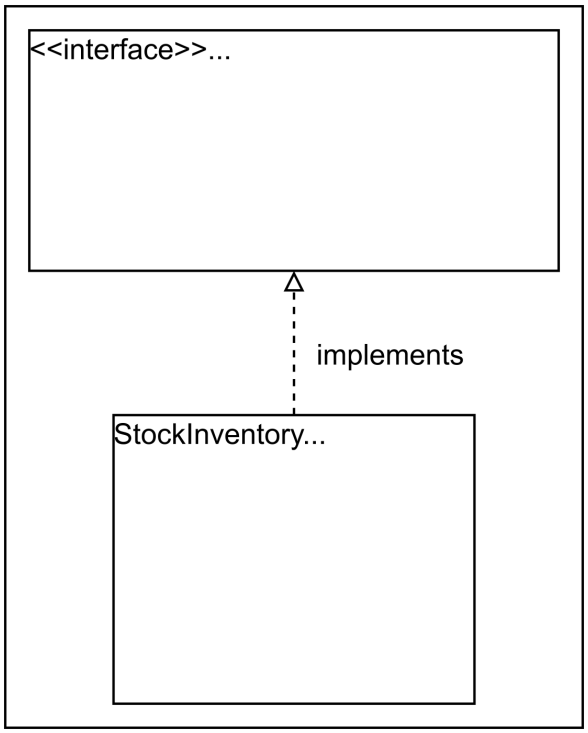


Figure 1.6: The class diagram of the Search interface and the StockInventory class

Stock position

All the stocks the user owns will be included in the `StockPosition` class.

Stock lot

A member may purchase various lots of the same stock on various dates. The `StockLot` class will represent these particular lots.

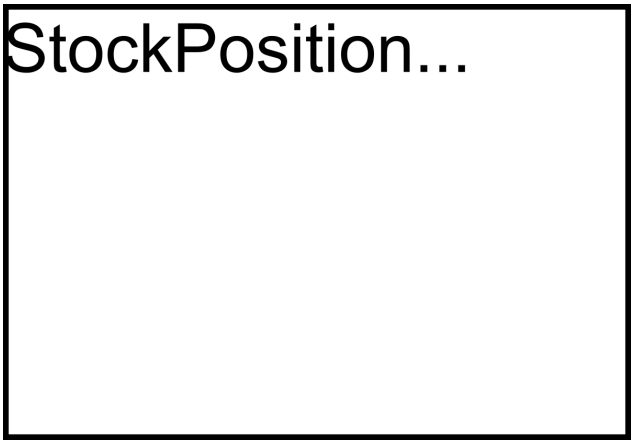


Figure 1.7: The class diagram of the StockPosition class

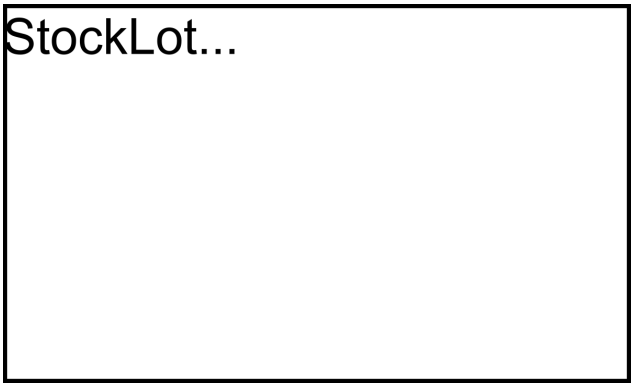


Figure 1.8: The class diagram of the StockLot class

R3: Users may own different lots of the same stock. If a user has purchased the same stock more than once, the system should be able to distinguish between several lots of it.

Order

Members can place stock trading orders to sell or acquire stock positions. There are four types of orders supported by the system:
`MarketOrder`**:** This allows customers to purchase or sell equities immediately at the market's going rate (current market price).
`LimitOrder`**:** A user may specify a price at which they wish to purchase or sell a stock.
`StopLossOrder`**:** An order to purchase or sell whenever the stock hits a specific price.
`StopLimitOrder`**:** The `StopLimitOrder` becomes a limit order to purchase or sell at the limit price, or better, if the stop price is reached.

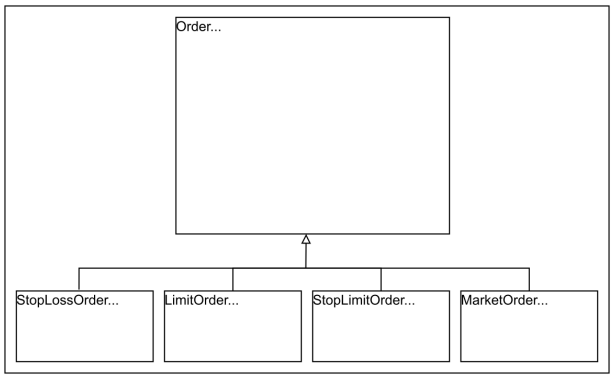


Figure 1.9: The class diagram of Order and its derived classes

R5: The system should allow the user to order the stock trade of the types given below:

- **Market order:** Buy or sell stocks at the current market price.
- **Limit order:** Buy or sell stocks at the price set by the user.
- **Stop-loss order:** Buy or sell stocks when they reach a certain price.
- **Stop-limit order:** Buy or sell stocks with a restriction on the limit price (maximum price to be paid, minimum price to be received, etc).

Order part

Multiple order parts might be used to complete an order. The `OrderPart` class contains the price, quantity, and execution date.

Deposit and withdraw money

The `DepositMoney` class represents a transfer of money from one party to another.
The `WithdrawMoney` class represents removing money from an account.

R6: The system should allow users to make deposits and withdrawals using checks, wire transfers, or electronic bank transfers.

Transfer money

Users should be able to deposit and withdraw money via check, wire, or electronic bank transfer.

R6: The system should allow users to make deposits and withdrawals using checks, wire transfers, or electronic bank transfers.

Figure 1.10: The class diagram of the OrderPart class

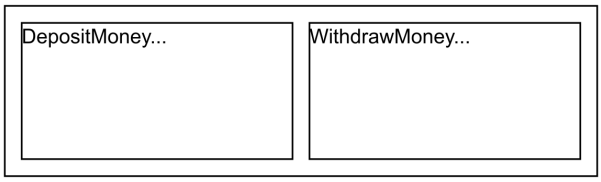


Figure 1.11: The class diagram of the DepositMoney and WithdrawMoney classes

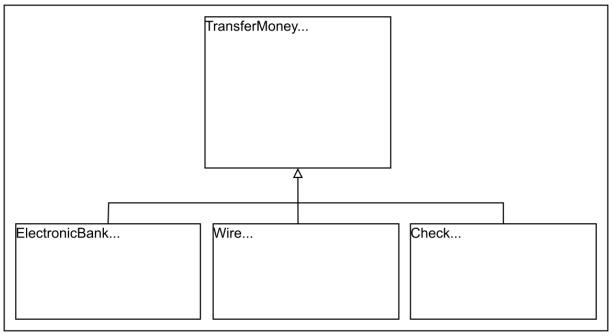


Figure 1.12: The class diagram of TransferMoney and its derived classes

Notification

`Notification` is an abstract class responsible for sending notifications when trade orders are executed. Every notification has an ID, creation date, and content. It can be an SMS or email notification. The`SmsNotification` class requires the member's phone number to send a notification. The `EmailNotification` class needs the member's email address to send a notification. The relationship diagram of these classes is shown here:

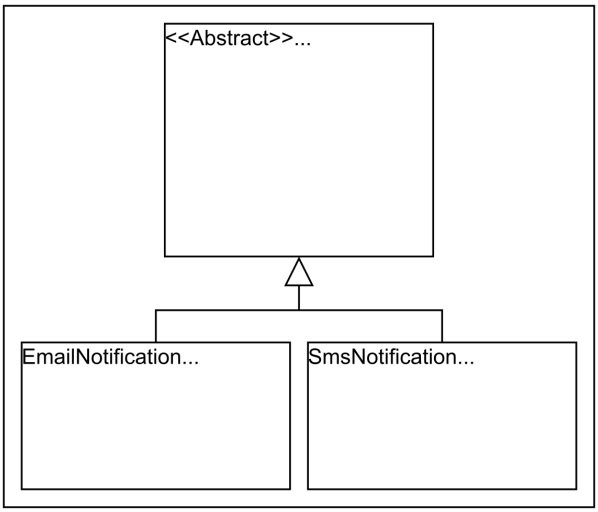


Figure 1.13: The class diagram of Notification and its derived classes

R4: The system should be able to notify users whenever a trade order is executed.

Stock exchange

The stock brokerage system will get all stocks from the stock exchange and their current pricing. The `StockExchange` class is responsible for creating orders for trading stocks on the stock exchange.

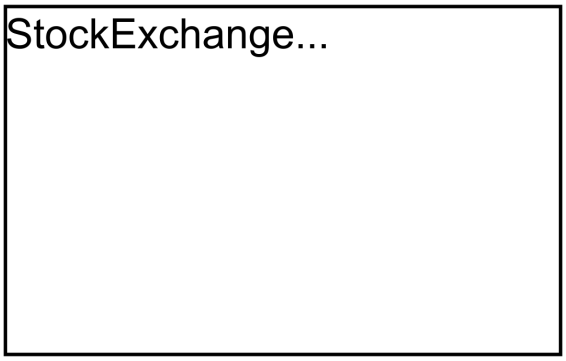


Figure 1.14: The class diagram of the StockExchange class

Enumerations and custom data types

The following provides an overview of the enumerations and custom data types used in this problem:

- `OrderStatus`: We need to create an enumeration to keep track of the status of the order, whether it is open, filled, partially filled, or canceled.

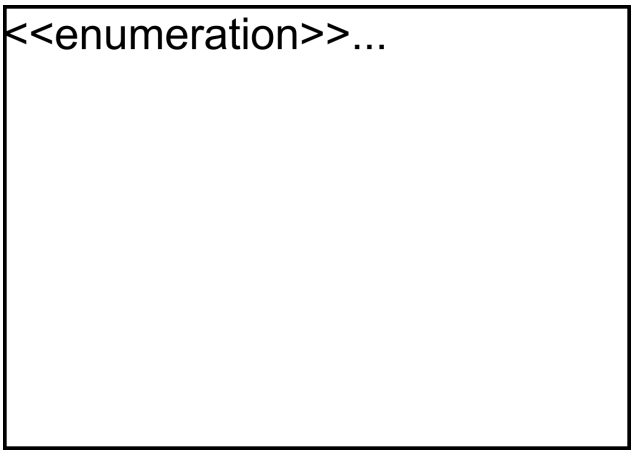


Figure 1.15: The OrderStatus enumerations

- `TimeEnforcementType`: We need to create an enumeration for the time enforcement type, whether it is good till canceled, fill or terminate, immediate or cancel, on the open, or on the close.

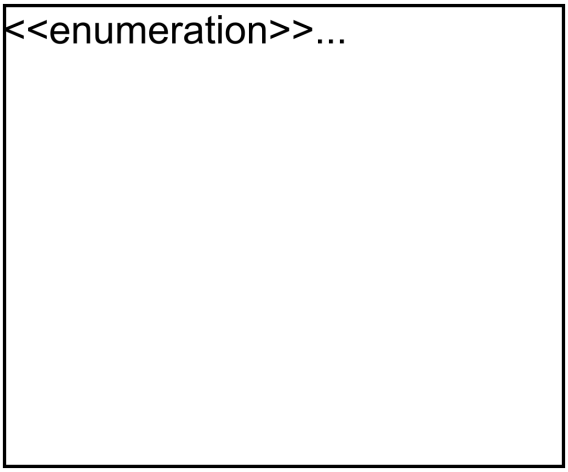


Figure 1.16: The TimeEnforcementType enumerations

- `AccountStatus`: We need to create an enumeration to keep track of the account's status, whether it is active, canceled, closed, blacklisted, or none.

- `ErrorCode`: We need to create an enumeration to represent the outcome of operations, indicating whether they were successful, failed due to insufficient funds, invalid inputs, or were rejected for other reasons.

Address We also need to create a custom data type, `Address`, to store the user's location.

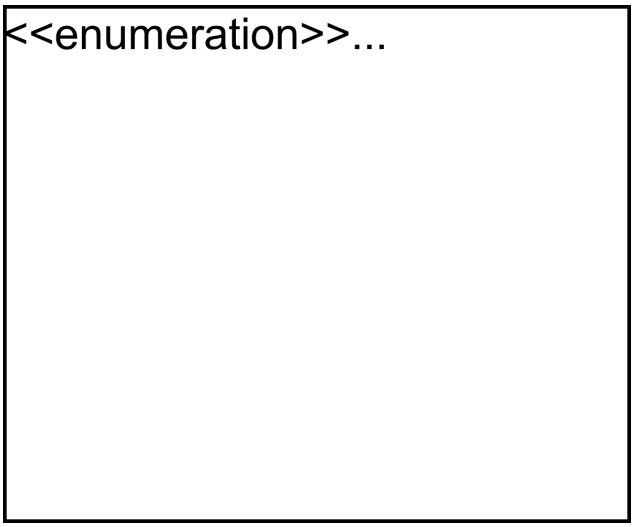


Figure 1.17: The AccountStatus enumerations

Figure 1.18: The class diagram of the Address class

4.2 Relationship between the classes

Now, we'll discuss the relationships between the classes we have defined above in our online stock brokerage system.

Association

The class diagram has the following association relationships:

One-way association

- The `StockInventory` class has a one-way association with `Watchlist` and`StockExchange`.
- The `Order` class has a one-way association with `Stock`, `StockExchange`, and `StockLot`.
- The `Account` class has a one-way association with `Order`, `DepositMoney`, and`WithdrawMoney`.
- The `StockPosition` classhas a one-way association with the `Order` class.

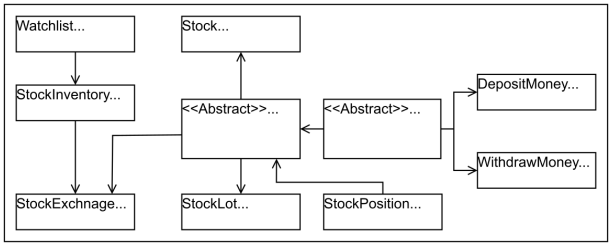


Figure 1.19: The one-way association relationship between classes

Two-way association

- The `Notification` class has a two-way association with the `Order` class.
- Both `Watchlist` and`StockPosition` have a two-way association with the `Account` class.

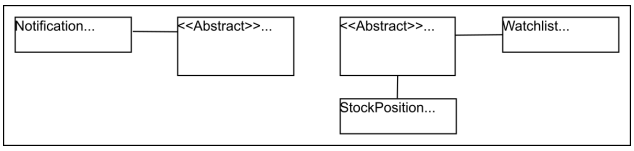


Figure 1.20: The two-way association relationship between classes

Composition

The class diagram has the following composition relationships:

- The `Order` classis composed of `OrderPart`.
- The `StockInventory` class is composed of `Stock`.
- The `StockPosition` class is composed of `StockLot`.

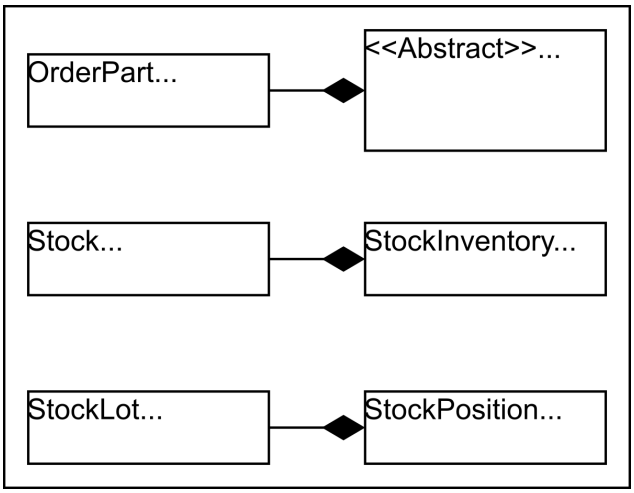


Figure 1.21: The composition relationship between classes

Inheritance

The following classes show an inheritance relationship:

- Both `Admin` and `Member` extend the `Account` class.
- The `MarketOrder`, `LimitOrder`, `StopLimitOrder`, and `StopLossOrder` classes extend the `Order` class.
- The `ElectronicBank`, `Wire`, and `Check` classes extend the `TransferMoney` class.
- The `SmsNotification` and `EmailNotification` classes extend the `Notification` class.
- The `StockInventory` class implements the `Search` interface.

  Note: We have already discussed the inheritance relationship between classes in the component section above one by one.

4.3 Class diagram of the online stock brokerage system

This section outlines the multiplicity (cardinality) relationships between the main classes in our online stock brokerage system. We explain the allowed number of instances on each side for each relationship and the real-world or design rationale behind the connection. Understanding these relationships is key to modeling how different entities interact and collaborate to support key workflows in the system.

Here is the complete class diagram for our online stock brokerage system:

4.4 Design pattern

To implement online stock brokerage's core features in a flexible and scalable way, we apply the object-oriented design patterns based on system behavior.

- The trading system handles different types of orders, such as market, limit, stop-loss, and stop-limit. We can use the Factory design pattern to manage the creation of these order types efficiently.
- The system processes payments through multiple channels, such as electronic bank, wire, or check. The Strategy design pattern can encapsulate these varying payment methods.
- Users or watch lists may want to be notified when a stock's price changes. W\sout{e} can use the Observer design pattern to model this real-time notification behavior.

Source	Target	Multiplicity	Description
StockExchange	Order	1 – 0..*	A stock exchange can receive multiple orders.
StockExchange	OrderPart	1 – 0..*	A stock exchange manages multiple order parts.
Watchlist	Stock	0..*	A watchlist can contain multiple stocks.
StockInventory	Stock	1 – 0..*	Inventory may manage multiple stocks.
Order	OrderPart	1 – 0..*	An order is composed of multiple order parts.
Notification	EmailNotification	0..*	Generalization/Inheritance.
Notification	SmsNotification	0..*	Generalization/Inheritance.
Account	Address	1 – 1	Each account has one address.
Account	DepositMoney	1 – 0..*	An account can have multiple deposit transactions.
Account	WithdrawMoney	1 – 0..*	An account can have multiple withdrawal transactions.
Admin	Account	1 – 1	Admin inherits from Account.
Member	Account	1 – 1	Member inherits from Account.
Member	StockPosition	1 – 0..*	A member may hold multiple stock positions.
Member	Order	1 – 0..*	A member can place multiple orders.
StockPosition	Stock	1 – 1	A stock position is related to one stock.
TransferMoney	ElectronicBank	1 – 1	Transfer via electronic bank.
TransferMoney	Wire	1 – 1	Transfer via wire.
TransferMoney	Check	1 – 1	Transfer via check.
StopLossOrder	Order	1 – 1	StopLossOrder inherits from Order.
LimitOrder	Order	1 – 1	LimitOrder inherits from Order.
StopLimitOrder	Order	1 – 1	StopLimitOrder inherits from Order.
MarketOrder	Order	1 – 1	MarketOrder inherits from Order.
StockLot	Order	1 – 1	Each stock lot is linked to one buying order.

Table 1.1: Stock Exchange System Entity Relationships

- Notifications can be sent via different channels, like email or SMS, and we might want to extend this with extra features like logging. To support this extensibility, we can use the Decorator design pattern.

- We know that only one `StockExchange` instance should be shared across the application. To ensure

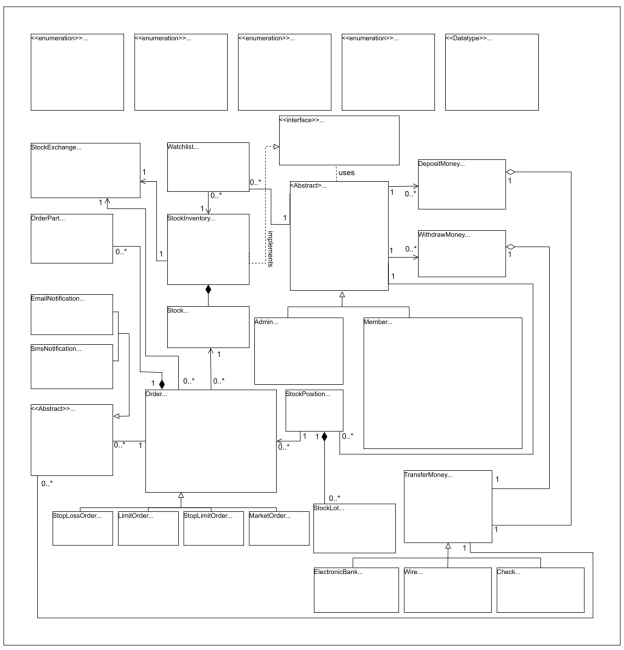


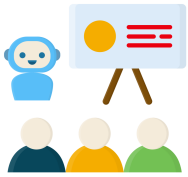
Figure 1.22: The class diagram of the online stock brokerage system

this, we can use the Singleton design pattern.

- We know that the process of sending a notification may follow a common structure but differ in implementation (email vs. SMS). We can use the Template Method design pattern to enforce this structure while allowing customization.
- We know that placing an order or sending a notification may need to be queued, logged, or executed later. We can use the Command design pattern to encapsulate these requests as objects.
- We know an order may consist of multiple parts, each with its own quantity and price, but we want to treat them as a single entity. To achieve this, we can use the Composite design pattern.
- Creating a complex order with multiple parts, statuses, and configurations can involve many steps. We can use the Builder design pattern to simplify and organize this process.

4.5 AI-powered trainer

At this stage, everything should be clear. If you encounter any confusion or ambiguity, feel free to utilize the interactive AI-enabled widget below to seek clarification. This tool is designed to help you strengthen your understanding of the concepts.



Component type 'PromptAI' not yet supported.

We have successfully designed the class diagram for the online stock brokerage system. Let 's move to the next lesson to see how we can construct the sequence diagram for the system.

5 Sequence Diagram for the Online Stock Brokerage System

Sequence diagrams are a great way to understand the interactions between different entities and objects in the system. There can be different sequence diagrams that we can create for the online stock brokerage system. In this lesson, we will create a sequence diagram for the following interaction:

- **Selling a stock:** A member wants to sell a stock on the stock market.

5.1 Selling a stock

The sequence diagram for selling a stock should have the following actors and objects that will interact with each other:

- **Actor:** `Member`
- **Objects:** `StockInventory` and `StockExchange`

Here are the steps involved in selling a stock:

1. A member selects a stock and its selling parameters: order type, quantity, time price limit, and time enforcement.
2. If the stock is available in the stock inventory:
   (a) The stock quantity is deducted from the stock inventory.
   (b) The stock order details will be sent to the stock exchange.
   (c) The stock exchange acknowledges it.
   (d) The member will be notified about the order placement.
3. Else if, enough stock is not available:
   (a) The member will be notified about stock unavailability.

Based on the order above, the sequence diagram for selling a stock in an online stock brokerage system is provided below.

Next, let's examine the activity diagrams for an online stock brokerage system to understand its control flow.

6 Activity Diagram for the Online Stock Brokerage System

Activity diagrams are a great way to visualize the flow of messages from one activity to another in the system. We can create different activity diagrams for our online stock brokerage system. In this lesson, we will create an activity diagram for the following activity.

6.1 Buying a stock

The states and actions involved in this activity diagram are listed below.

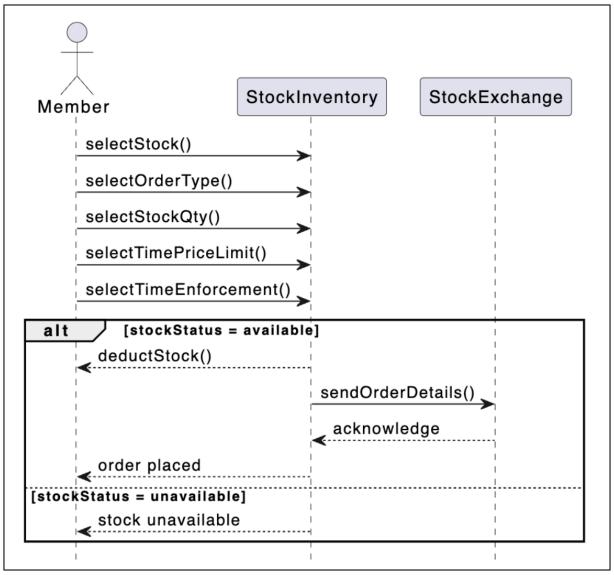


Figure 1.23: The sequence diagram for selling a stock

States

Initial state: The member selects a stock to buy.
Final state: There are two final states present in this activity diagram, shown below:

- The member successfully buys a stock from the online stock brokerage system.
- Due to the unavailability of a sufficient deposit, members cannot buy the stocks.

Actions

The member selects the stock to buy, the order type, quantity, time price limit, and time enforcement. The system then checks if the member has a sufficient deposit in their account. If the funds are available, the system deducts the amount, sends the order details to the stock exchange, and, upon receiving acknowledgment, notifies the member of the successful order. The purchased stocks are then added to the member's inventory.
Based on the order above, the activity diagram for buying a stock at the online stock brokerage system is provided below:

> **Note:** This activity is limited to cases where the stock is successfully placed only if the deposit is available.

We've looked at the activity diagram of our online stock brokerage system. In the next lesson, we will present the code for our designed classes in some of the most popular languages.

7 Code for the Online Stock Brokerage System

We've reviewed different aspects of the online stock brokerage system and observed the attributes attached to the problem using various UML diagrams. Let 's explore the more practical side of things, where we will work on implementing the online stock brokerage system using multiple languages. This is usually the last step in an object-oriented design interview process.

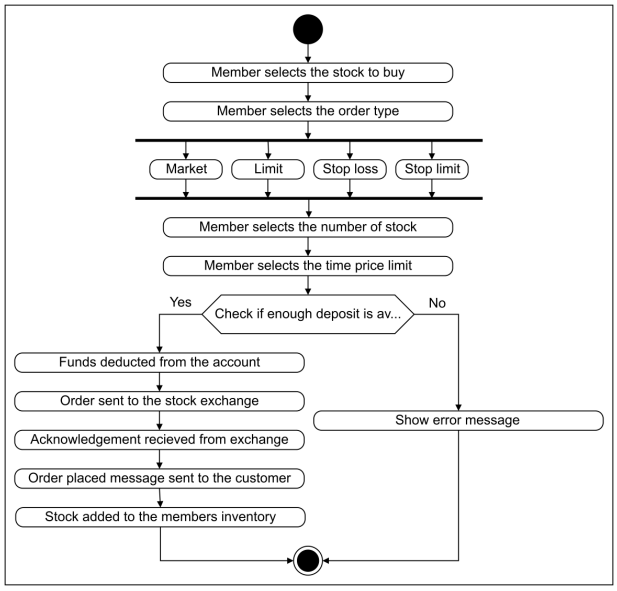


Figure 1.24: The activity diagram for buying a stock

We have chosen the following languages to write the skeleton code of the different classes present in the online stock brokerage system:

- Java
- C#
- Python
- C++
- JavaScript

If you want to skip directly to the implementation and see the complete workflow, simply scroll down or click Executable Code: Online Stock Brokerage System.

7.1 Online stock brokerage system classes

This section will provide the skeleton code of the classes designed in the class diagram lesson.

> **Note:** For simplicity, we are not defining getter and setter functions. The reader can assume that all class attributes are private, accessed through their respective public getter methods, and modified only through their public method functions.

Enumerations and custom data type

First, we will define all the enumerations required in the stock brokerage system. According to the class diagram, four enumerations are used in the system, i.e., `OrderStatus`, `TimeEnforcementType`, `AccountStatus`,and `ErrorCode`. The code to implement these enumerations and custom data types is as follows:

> **Note:** JavaScript does not support enumerations, so we will use the `Object.freeze()` method as an alternative that freezes an object and prevents further modifications.

Definition of enums and custom data types

```
// Enumeration
enum OrderStatus {
  OPEN,
  FILLED,
  PARTIALLY_FILLED,
  CANCELED
}

enum TimeEnforcementType {
  GOOD_TILL_CANCELED,
  FILL_OR_TERMINATE,
  IMMEDIATE_OR_CANCEL,
  ON_THE_OPEN,
  ON_THE_CLOSE
}

enum AccountStatus {
  ACTIVE,
  CLOSED,
  CANCELED,
  BLACKLISTED,
  NONE
}

public enum ErrorCode {
    SUCCESS,
    INSUFFICIENT_FUNDS,
    INVALID_STOCK,
    ORDER_REJECTED
}

// Custom Address data type class
public class Address {
  private int zipCode;
  private String address;
  private String city;
  private String state;
  private String country;
}
```

Account

The `Account` class will be an abstract class with the actors, `Admin` and `Member`, as child classes. The definition of these classes is given below:

Account and its child classes

```
// Enumeration
enum OrderStatus {
  OPEN,
  FILLED,
  PARTIALLY_FILLED,
  CANCELED
}

enum TimeEnforcementType {
  GOOD_TILL_CANCELED,
  FILL_OR_TERMINATE,
  IMMEDIATE_OR_CANCEL,
  ON_THE_OPEN,
  ON_THE_CLOSE
}

enum AccountStatus {
  ACTIVE,
  CLOSED,
```

```
  CANCELED,
  BLACKLISTED,
  NONE
}

public enum ErrorCode {
    SUCCESS,
    INSUFFICIENT_FUNDS,
    INVALID_STOCK,
    ORDER_REJECTED
}

// Custom Address data type class
public class Address {
  private int zipCode;
  private String address;
  private String city;
  private String state;
  private String country;
}
```

Watch list and stock

A `Watchlist` class is a list of stocks that an investor monitors to profit from price drops. The `Stock` class is an equity or security that represents a portion of the issuing company's ownership.

The Watchlist and the Stock classes

```
public class Watchlist {
  private String name;
  private List<Stock> stocks;
}

public class Stock {
  private String symbol;
  private double price;
}
```

Search and stock inventory

The `StockInventory` class implements the `Search` interface.

The Search interface and the StockInventory class

```
public interface Search {
    // Interface method (does not have a body)
    public Stock searchSymbol(String symbol);
}

public class StockInventory implements Search {
  private String inventoryName;
  private Date lastUpdate;
  public Stock searchSymbol(String symbol);
  public boolean deductStock(String symbol, double quantity);
  public boolean sendOrderDetails(Order order);
}
```

Stock position and stock lot

All the stocks the user owns will be included in the `StockPosition` class. A member may purchase various lots of the same stock on various dates. The `StockLot` class will represent these particular lots.

The StockPosition and StockLot classes

```
public class StockPosition {
  private String symbol;
  private double quantity;
}

public class StockLot {
  private String iotNumber;
  private Order buyingOrder;

  public double getBuyingPrice();
}
```

Order

Members can place stock trading orders to sell or acquire `StockPosition`.

OrderPart, Order, and its derived classes

```
public class OrderPart {
  private double price;
  private double quantity;
  private Date executedAt;
}

// Order is an abstract class
public abstract class Order {
  private String orderNumber;
  public boolean isBuyOrder;
  private OrderStatus status;
  private TimeEnforcementType timeEnforcement;
  private Date creationTime;
  private HashMap<int, OrderPart> parts;

  public void setStatus(OrderStatus status);
  public boolean saveInDatabase();
  public void addOrderParts(OrderParts parts);
}

public class LimitOrder extends Order {
  private double priceLimit;
}

public class StopLimitOrder extends Order {
  private double priceLimit;
}

public class StopLossOrder extends Order {
  private double priceLimit;
}

public class MarketOrder extends Order {
  private double priceLimit;
}
```

Transfer money

Members should be able to deposit and withdraw money via `Check`, `Wire`, or `ElectronicBank` transfer.

TransferMoney and its derived classes

```
// TransferMoney is an abstract class
public abstract class TransferMoney {
  private int id;
  private Date creationDate;
  public int fromAccount;
  private int toAccount;

  public abstract boolean initiateTransaction();
}

public class ElectronicBank extends TransferMoney {
  private String bankName;

  public boolean initiateTransaction();
}

public class Wire extends TransferMoney {
  private int wire;

  public boolean initiateTransaction();
}

public class Check extends TransferMoney {
  private String checkNumber;

  public boolean initiateTransaction();
}

public class DepositMoney {
  private int transactionId;

  public boolean initiateTransaction();
}

public class WithdrawMoney {
  private int transactionId;
  public boolean initiateTransaction();
}
```

Notification

The `Notification` class is another abstract class responsible for sending notifications to the users, with the `SmsNotification` and `EmailNotification` classes as its child classes. The implementation of this class is shown below:

Notification and its derived classes

```
public abstract class Notification {
    private String notificationId;
    private Date creationDate;
    private String content;

    public abstract boolean sendNotification();
}

public class SmsNotification extends Notification {
    private int phoneNumber;

    public boolean sendNotification();
```

```
}

public class EmailNotification extends Notification {
    private String email;

    public boolean sendNotification();
}
```

Stock exchange

The stock brokerage system will get all the stocks and their current pricing from the `StockExchange` class.

The StockExchange class

```
public class StockExchange {
  // The StockExchange is a singleton class that ensures it will have only one active instance at a
      time
  private static StockExchange instance = null;
  private StockExchange() {}
  public static StockExchange getInstance();
  public boolean placeOrder(Order order);
  public boolean acknowledge(Order order);
}
```

7.2 Executable code: Online stock brokerage system

Below is a fully self-contained, runnable program in Java, C#, C++, Python, and JavaScript that demonstrates the core workflows of the online stock brokerage system. The main driver code of the system resides in the `Driver.java`, `Driver.cs`, `Driver.py`, `Driver.cpp`, and `Driver.js` for all respective languages. You can click the "Run" button to execute the codes.

Online Stock Brokerage System

```
import java.util.*;

public class Driver {
    public static void main(String[] args) {
        System.out.println("========== ONLINE STOCK BROKERAGE SYSTEM ==========");

        // ---------------------------------------------------
        // SYSTEM INITIALIZATION
        // ---------------------------------------------------
        System.out.println("
--- SYSTEM INITIALIZATION ---");

        // Create and configure essential system components
        StockInventory inventory = new StockInventory();
        StockExchange exchange = StockExchange.getInstance();

        // Create and initialize member
        Member member = new Member();
        member.setName("Alice");
        member.setAvailableFundsForTrading(10000.0);

        System.out.println("[INFO] Member Initialized: " + member.getName());
        System.out.println("[INFO] Available Funds: $" + member.getAvailableFundsForTrading());

        // ---------------------------------------------------
        // SCENARIO 1: SEARCHING AND SELECTING A STOCK
        // ---------------------------------------------------
        System.out.println("
```

```
--- SCENARIO 1: SEARCHING AND SELECTING A STOCK ---");

        String symbolToSearch = "AAPL";
        System.out.println("[STEP] Searching for stock with symbol: " + symbolToSearch);
        Stock stock = inventory.searchSymbol(symbolToSearch);

        // Fallback if stock not found
        if (stock == null) {
            System.out.println("[WARNING] Stock not found in inventory. Creating mock stock data.");
            stock = new Stock();
            stock.setSymbol(symbolToSearch);
            stock.setPrice(180.50);
        }

        System.out.println("[RESULT] Stock Found: " + stock.getSymbol() + " @ $" + stock.getPrice())
             ;

        // Member selects the stock
        member.selectStock(stock.getSymbol());
        double quantity = 10;
        System.out.println("[ACTION] Selected Quantity: " + quantity);

        // ---------------------------------------------------
        // SCENARIO 2: PLACING A LIMIT ORDER
        // ---------------------------------------------------
        System.out.println("
--- SCENARIO 2: PLACING A LIMIT ORDER ---");

        Order order = new LimitOrder();
        order.setIsBuyOrder(true);
        order.setOrderNumber("ORD123");
        ((LimitOrder) order).setPriceLimit(stock.getPrice() - 5); // Buy at $175.50
        order.setTimeEnforcement(TimeEnforcementType.GOOD_TILL_CANCELED);
        order.setCreationTime(new Date());

        System.out.println("[ORDER] Type: Limit Order");
        System.out.println("[ORDER] Order Number: " + order.getOrderNumber());
        System.out.println("[ORDER] Buy Order: " + order.isBuyOrder);
        System.out.println("[ORDER] Limit Price: $" + ((LimitOrder) order).getPriceLimit());
        System.out.println("[ORDER] Time Enforcement: " + order.getTimeEnforcement());
        System.out.println("[ORDER] Created On: " + order.getCreationTime());

        // Place order through stock exchange
        boolean orderPlaced = exchange.placeOrder(order);
        inventory.sendOrderDetails(order);

        if (orderPlaced) {
            System.out.println("[SUCCESS] Order placed successfully: " + order.getOrderNumber());
        } else {
            System.out.println("[FAILED] Order placement failed.");
        }

        // Acknowledge order by exchange
        boolean ack = exchange.acknowledge(order);
        System.out.println("[EXCHANGE] Order Acknowledged: " + ack);

        // ---------------------------------------------------
        // SCENARIO 3: SENDING ORDER NOTIFICATION
        // ---------------------------------------------------
        System.out.println("
--- SCENARIO 3: SENDING ORDER NOTIFICATION ---");

        Notification notification = new EmailNotification();
        notification.setCreationDate(new Date());
        notification.setContent("Your order " + order.getOrderNumber() + " has been received.");
        ((EmailNotification) notification).setEmail("alice@example.com");

        boolean sent = notification.sendNotification();
        System.out.println("[NOTIFICATION] Email sent to " + ((EmailNotification) notification).
            getEmail() + ": " + sent);

        // ---------------------------------------------------
```

```
        // SESSION END
        // ---------------------------------------------------
        System.out.println("
==================================================");
        System.out.println(" Thank you for trading with us, " + member.getName());
        System.out.println("==================================================");
    }
}
```

What does this code show

- **System initialization:** Initializes the core components of the online stock brokerage system, including the stock inventory, stock exchange (as a Singleton instance), and a member named Alice with a starting trading fund of $10,000. This sets up the environment for simulating stock trading operations.
- **Scenario 1 (Stock search and selection):** Alice searches for a stock using its symbol ("AAPL"). If the stock is found in the inventory, its details are retrieved; otherwise, a mock stock is created. Alice selects this stock and specifies the quantity to trade, simulating a typical user action before placing an order.
- **Scenario 2 (Placing a limit order):** Alice creates a buy limit order for the selected stock, specifying a limit price below the current market value and setting the order enforcement to "Good Till Canceled." The order is submitted to the stock exchange, and its details are sent to the inventory. The system confirms whether the order was placed successfully and receives an acknowledgment from the exchange.
- **Scenario 3 (Notification of order placement):** Once the order is placed and acknowledged, the system generates an email notification for Alice, confirming that her order has been received. This demonstrates how real-time notifications keep users informed of important transaction events.

  Hint: Try customizing the code!
  This code is designed for experimentation, not just reading. You can edit, extend, or modify any part of the example.

In this chapter, we've explored the complete design of an online stock brokerage system. We 've also looked at how a basic system can be visualized using various UML diagrams and designed using object-oriented principles and design patterns.

8 Summary: Designing an Online Stock Brokerage System

Now that you've completed the online stock brokerage system case study, let's take a moment to reflect on and consolidate what we've learned. We 'll revisit the key system requirements, identify the core classes along with their responsibilities and relationships, and highlight the major design principles applied. We 'll also examine how objects interact within the system and walk through the overall workflow to understand how the components come together to achieve the desired functionality.

8.1 Key requirements

Below are the primary functional and nonfunctional requirements that define the scope of the online stock brokerage system:

1. The system must allow users to easily trade in stocks (buy or sell stocks).
2. Users can maintain multiple watchlists, each with different stock quotes.

3. The system should track different lots of the same stock for each user, distinguishing between several purchases of the same stock.
4. Users must receive notifications whenever trade orders are executed.
5. The system must support various order types: market order, limit order, stop-loss order, and stop-limit order.
6. Deposits and withdrawals should be supported via check, wire, and electronic bank transfers.

8.2 System actors

The following primary actor interacts with the system:

- **Member:** A user who can register, log in, manage their account, search stocks, manage watch lists, place and cancel orders, perform deposits/withdrawals, and receive notifications.

8.3 Important classes and relationships

The system is composed of several key classes, each playing a specific role and connecting to others through various relationships:

8.4 Design highlights

The online stock brokerage system is thoughtfully architected, employing best practices for clarity, maintainability, and scalability. Below are key aspects of the design:

Design approach

The system is designed using a bottom-up methodology. Core components and domain entities (such as stocks, orders, and accounts) are identified first, and higher-level workflows and relationships are built on these foundations.

Design principles

The system applies core SOLID principles, ensuring a robust and extensible design:

- **Single Responsibility principle (SRP):** Each class (e.g., `Order`, `Notification`, `StockInventory`) is responsible for a single aspect of the system, such as order management, notifications, or stock data.
- **Open/Closed principle (OCP):** The order and notification hierarchies are open for extension (new order or notification types can be added) but closed for modification of existing code.
- **Liskov Substitution principle (LSP):** Subclasses such as `MarketOrder`, `LimitOrder`, etc., can be used interchangeably via the `Order` base class.
- **Interface Segregation principle (ISP):** `StockInventory` implements the `Search` interface, ensuring classes expose only relevant behavior.
- **Dependency Inversion principle (DIP):** High-level modules interact with abstractions (e.g., `Order`, `Notification`) rather than concrete implementations.

Design patterns

The design makes use of classic object-oriented design patterns for extensibility and clarity:

- **Factory pattern:** Used for creating various orders and notifications without exposing instantiation logic to the client.

Class	Description	Important Relationships
Account	Abstract base for user accounts, storing details and status.	Inherited by Admin, Member; associated with Order, DepositMoney, WithdrawMoney, Address.
Member	Represents a typical system user with trading and portfolio features.	Inherits from Account; associated with Watchlist, StockPosition, Order.
Admin	Represents admin users with elevated permissions.	Inherits from Account.
Watchlist	List of stocks tracked by a member.	Associated with Member; contains multiple Stock.
Stock	Represents a stock/equity with a symbol, price, etc.	Contained in StockInventory, Watchlist, StockPosition.
StockInventory	Manages and updates available stock data; implements Search.	Associated with StockExchange; contains multiple Stock.
StockExchange	External or internal stock market source for real-time data and order execution.	Receives Order, manages OrderPart, associated with StockInventory.
StockLot	Represents individual lots of the same stock purchased at different times/prices.	Part of StockPosition; linked to one Order.
Order	Abstract base for orders (market, limit, stop-loss, stop-limit).	Inherited by order types; composed of multiple OrderPart; associated with Stock, StockLot, StockExchange.
OrderPart	Represents partial or batch execution of an order.	Part of Order; managed by StockExchange.
DepositMoney	Represents a deposit transaction.	Associated with Account.
WithdrawMoney	Represents a withdrawal transaction.	Associated with Account.
TransferMoney	Abstract base for different transfer methods.	Inherited by ElectronicBank, Wire, and Check.
Search	Provides searching capabilities for stocks.	Implemented by StockInventory.

Table 1.2: Stock Exchange System Classes and Their Relationships

- **Observer pattern:** This pattern sends members real-time notifications (via `Notification`) when trades or transactions occur.
- **Strategy pattern:** Applied to implement different order types, allowing interchangeable order creation and execution strategies.
- **Decorator Design pattern:** The system should support sending notifications via multiple channels (such as email or SMS) and allow for extensibility using the Decorator design pattern.
- **Singleton Design pattern:** The Singleton design pattern enforces that there should only be one shared instance of the `StockExchange` throughout the application.

8.5 Object interactions

The system's dynamic behavior is captured through object collaboration during key activities:

- **Placing an order:** The `Member` selects a stock and order details, which are sent to the `StockInventory` to verify availability and retrieve current prices.
- **Order creation:** If sufficient stock (for selling) or funds (for buying) are available, the system creates an Order object (of the appropriate type).
- **Stock adjustment:** The system deducts the specified quantity from the member's `StockPosition` or `StockInventory` for sales. For purchases, funds are deducted from the member's account.
- **Sending to exchange:** The `Order` details are forwarded to the `StockExchange`, which processes the transaction and returns an acknowledgment.
- **Notification:** Upon successful execution or failure, a `Notification` (either `SmsNotification` or `EmailNotification`) is generated and sent to the `Member` to inform them of the result.
- **Transaction handling:** For deposits or withdrawals, a `DepositMoney` or `WithdrawMoney` object is created, the transaction is processed, and a corresponding notification is sent to the member.

8.6 System workflow

The system supports several end-to-end workflows, which guide users and administrators through the main processes:

- **Stock purchase:** Member selects a stock to buy, enters order details, and submits. The system checks for sufficient deposit, deducts funds, creates an order, sends it to the stock exchange, and notifies the user of the outcome.
- **Stock sale:** Member selects a stock to sell, configures order parameters, and submits. The system verifies stock availability, deducts the quantity, forwards the order to the exchange, and notifies the user.
- **Account management:** Members or admins can create, update, or cancel accounts and reset passwords.
- **Watch list management:** Members can create, update, and view watchlists, tracking desired stocks and monitoring changes.
- **Deposits/Withdrawals:** Members deposit or withdraw funds via supported methods; the system processes transactions and sends notifications.
- **Order notifications:** The system automatically notifies members of order executions, cancellations, or failures, as well as deposit/withdrawal status.

9 Key Insights and Tips: Designing a Stock Brokerage System

Congratulations on finishing the online stock brokerage system case study! In this lesson, we'll reinforce your learning by pointing out common design mistakes to avoid and sharing effective strategies for tackling interview questions. You 'll also find a quick quiz to test your understanding and a list of related case studies to help you deepen your grasp of key object-oriented design principles.

9.1 Common mistakes

Avoiding these common mistakes will help you create a more reliable and scalable online stock brokerage system:

- **Misunderstanding user roles and permissions:** Confusing the roles and permissions of `Member` and `Admin` can lead to security issues or unauthorized access. Ensure that `Admin` and `Member` roles have well-defined, separate permissions. `Admin` should manage users and the system, while a `Member` should only access and modify their data.
- **Handling multiple order types incorrectly:** Mismanaging the implementation of different order types, such as market, limit, stop-loss, and stop-limit orders, can cause incorrect stock trades or failures. Implement each order type as a subclass of a base`Order` class, ensuring specific behaviors (e.g., price validation) are handled separately for each type.
- **Not implementing real-time data updates:** Failing to keep stock data up-to-date can cause synchronization issues, leading to outdated or incorrect trading information. Use the `StockExchange` class to pull real-time data and update the `StockInventory` at regular intervals or in real time as needed.
- **Inefficient notification handling:** Not sending timely notifications or poorly managing notification types (SMS, email) can frustrate users. Use the Observer pattern to send real-time notifications, ensuring users receive accurate updates about their trade status, account changes, and transaction alerts.
- **Lack of scalability consideration:** Not designing the system to handle increasing users or trades can lead to performance bottlenecks as the system grows. To ensure the system can handle growth, consider scalability using modular components, optimizing queries, and possibly incorporating a microservices architecture.

9.2 Interview tips

Understand key design patterns, system components, and real-world applications to prepare for your interview and confidently discuss your approach.

9.3 Test your understanding

Now that you understand OOD principles well, solve the quiz below to test your knowledge.

Online Stock Brokerage System

Question 1: How would you allow your brokerage system to integrate with multiple external stock exchanges, each with its own API, without rewriting the main business logic?

- **[CORRECT]** Use the Adapter pattern to wrap each exchange's API and expose a unified interface.
  Explanation: This enables pluggable integration and keeps core trading logic unchanged.
- Copy-paste the integration code everywhere.
- Restrict to a single exchange.
- Only use synchronous calls for all APIs.

Question 2: What design would you use to allow users to create complex, customizable order types in the future (e.g., multi-condition triggers)?

Tip	Explanation
Understanding system components	Be prepared to discuss the stock brokerage system's modular design. Understand how components like StockInventory, StockPosition, and Order interact to form a seamless user experience.
Explain design patterns	Highlight design patterns like Singleton (for StockExchange), Observer (for notifications), and Factory (for creating different order types). Explain why these patterns are chosen.
Show awareness of error handling and transaction integrity	Detail how the system will handle failures in trade execution, notification delivery, or account transactions, ensuring system reliability and data consistency.
Highlight transaction management and consistency	Discuss how you'll maintain data consistency and handle atomicity for trades, deposits, and withdrawals. Reference database transactions, rollback strategies, or distributed transaction management if relevant.
Talk about user experience	Include how real-time updates, clear error messages, and timely notifications enhance the overall user experience for both Member and Admin.

Table 1.3: Stock Exchange System Design Tips and Explanations

- Hardcode every new order type in the `Order` class.
- **[CORRECT]** Leverage the Composite pattern to allow orders to be composed of multiple simpler conditions.

  Explanation: The Composite pattern allows the system to combine simple and complex order logic flexibly.
- Use only basic `if-else` logic in order of execution.
- Only allow market orders.

Question 3: Which design pattern best fits the requirement for sending users different notifications (SMS, email) in response to various system events (e.g., order filled, deposit made)?

- Singleton
- Adapter
- **[CORRECT]** Observer

  Explanation: The Observer pattern allows the system to notify multiple subscribers (SMS, email, etc.) whenever an event occurs, supporting extensible and decoupled notification delivery.
- Proxy

Question 4: What is the main benefit of using interfaces (such as a `Search` interface) in the design of this system?

- To ensure only one implementation

- **[CORRECT]** To enable polymorphism and allow flexible, interchangeable implementations
  Explanation: Interfaces promote loose coupling and flexibility, allowing different classes to implement search functionality as needed.
- To make the code harder to understand
- To speed up execution

Question 5: Imagine you want to track every order placed, modified, or canceled for auditing. What OOD technique or pattern is most suitable?

- Use global variables to record changes.
- **[CORRECT]** Implement an event sourcing approach to log all order events as immutable records.
  Explanation: Event sourcing captures the full history, making audits, rollbacks, and debugging possible.
- Overwrite the order every time without history.
- Only store the current state in the database.

9.4 Related case studies

The following case studies will help reinforce concepts used in the online stock brokerage system:

- **Design of a library management system:** This focus is on user roles, inventory management, notification patterns, and modular entity design.
- **Design of a vending machine:** Emphasizes extensible design for adding new product types, payment methods, and maintenance workflows.
- **Design of a parking lot system:** This course focuses on designing a parking lot system that handles multiple types of vehicles, parking spaces, and payments.
- **Design of an online bookstore:** This case study covers the design of an online bookstore, focusing on catalog management, user accounts, and order processing.
- **Design of an e-commerce marketplace:** Covers catalog management, multiple user roles (buyer, seller, admin), real-time inventory updates, order processing, payments, and notifications, mapping closely to user accounts and trade/order processing.